


# ECHOGENESIS: A LONGITUDINAL, DEGRADATION-ROBUST FRAMEWORK FOR VOICE PRESERVATION IN SPEECH LOSS

**S. UPENDIRAN – 24619230**

**SAIYUJ S – 24619247**

**B. LOKESH – 24619206**

**VELS INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)**

**MINI PROJECT REPORT**

Submitted to the

**SCHOOL OF ENGINEERING**

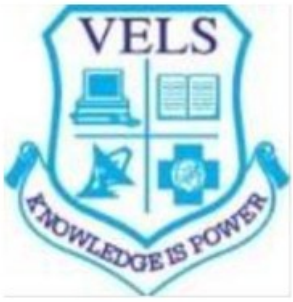
**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING**

in Partial fulfillment of the requirements for the award of the degree of

**BACHELOR OF TECHNOLOGY**

**IN**

**COMPUTER SCIENCE AND ENGINEERING**




**ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING**

**VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)**

**CHENNAI – 600 117**

**OCTOBER 2026**

## BONAFIDE CERTIFICATE

Certified that this project report "**ECHOGENESIS: A LONGITUDINAL, DEGRADATION-ROBUST FRAMEWORK FOR VOICE PRESERVATION IN SPEECH LOSS**", is the Bonafide work of **S. UPENDIRAN – 24619230**, **SAIYUJ S – 24619247**, **B. LOKESH – 24619206** who carried out the project under my supervision. Certified further, that to the best of my knowledge the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE OF THE HOD

**Dr. K. Ulaga Priya M.E., Ph.D**
Associate Professor & Head
Department AI&ML
School of Engineering
VISTAS
Chennai

SIGNATURE OF THE SUPERVISOR

**Dr. G. REVATHY M.E., Ph.D**
Assistant Professor
Department of AI&ML
School of Engineering
VISTAS
Chennai

Submitted for the Project Work and Viva – Voce examination held on ..... at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER

EXTERNAL EXAMINER

## ACKNOWLEDGEMENT

It is certainly a great delight and proud privilege to acknowledge the help and support we received from the positive minds around us in making this venture a successful one. The infrastructural support with all kinds of lab facilities has been a motivating factor in the completion of this project work, all because of our **FOUNDER & CHAIRMAN, Shri. Dr. ISHARI K. GANESH**, and it is with great pleasure that we take this opportunity to thank him. We are grateful to our **HEAD OF THE DEPARTMENT, Dr. K. Ulaga Priya**, for her continuous support for the project, from initial advice and contacts in the early stages of conceptual inception, and through on-going advice and encouragement to this day. We extend our warmest thanks to our **INTERNAL GUIDE, Dr. G. REVATHY, Assistant Professor/AI&ML**, who has patiently guided us and provided invaluable advice and help. We would like to express our greatest gratitude to our Staff members, Librarian, and Non-teaching staff members of the Computer Centre, who have helped and supported us throughout. Last but not the least, we wish to thank our parents and our friends for their individual support and interest, who have inspired us and encouraged us to go our own way; without them we would be unable to complete this project.

**S. UPENDIRAN – 24619230**

**SAIYUJ S – 24619247**

**B. LOKESH – 24619206**

| CHAPTER NO. | TABLE OF CONTENT | PAGE NO. |
|---|---|---|
| | **ABSTRACT** | i |
| | **LIST OF FIGURES** | ii |
| | **LIST OF TABLES** | ii |
| | **ABBREVIATIONS** | iii |
| **1** | **INTRODUCTION** | 1 |
| | 1.1 Overview of the project | 1 |
| | 1.2 Scope and objective | 2 |
| **2** | **LITERATURE REVIEW** | 5 |
| | 2.1 Literature Overview | 5 |
| | 2.2 Literature Survey | 5 |
| **3** | **SYSTEM DESIGN** | 10 |
| | 3.1 Existing System | 10 |
| | 3.2 Proposed System | 10 |
| | 3.3 System Architecture | 11 |
| | 3.4 Hardware Requirements | 12 |
| | 3.5 Software Requirements | 13 |
| **4** | **APPROACH AND EXECUTION** | 15 |
| | 4.1 Methodology | 15 |
| | 4.1.2 Evaluation metrics | 16 |
| | 4.2 Implementation | 18 |
| | 4.2.1 Voice Input & Preprocessing Module | 18 |
| | 4.2.2 Speaker Encoder & Identity Modeling Module | 19 |
| | 4.2.3 Longitudinal Voice Bank Module | 19 |
| | 4.2.4 Few-Shot TTS & Chunked Synthesis Module | 19 |
| | 4.2.5 Web Interface & API Module | 20 |
| | 4.3 Source Code | 21 |
| **5** | **RESULT ANALYSIS** | 25 |
| **6** | **CONCLUSION AND FUTURE WORK** | 27 |
| | 5.1 Conclusion | 27 |
| | 5.2 Future Work | 27 |
| | **REFERENCES** | 28 |

## ABSTRACT

EchoGenesis is a voice preservation solution for people whose speech changes because of conditions such as amyotrophic lateral sclerosis (ALS), Parkinson's disease, stroke, cancer treatment or laryngectomy. Current voice cloning and text-to-speech (TTS) systems are mostly built around healthy speech and do not sufficiently consider the longitudinal changes in a person's voice. EchoGenesis is capable of preserving the unique voice of a speaker even when his or her speech becomes progressively worse. The system uses a degradation-robust speaker encoder that identifies the speaker from damaged speech samples, and it compares more recent recordings with those recorded earlier, while the speaker still had a healthy voice. Natural speech is then generated from a limited number of voice samples through a few-shot text-to-speech model, and prosody-aware chunked synthesis is used for longer sentences.

The system is evaluated using speaker similarity, word error rate and a degradation-robustness curve. A proof-of-concept embedding-drift study shows that the drift signal increases with the degradation level and also reveals variability between different recording sessions. A working web application with a FastAPI backend, quick emergency and common phrases, emotion-controlled speech, a longitudinal voice bank and a caregiver view has been implemented to demonstrate the framework.



**Keywords:** Voice Cloning, Artificial Intelligence, Deep Learning, Amyotrophic Lateral Sclerosis, Throat Cancer, Speech Preservation, Assistive Technology, Personalized Voice Model, Word Error Rate.

## LIST OF FIGURES


| FIGURE NO | FIGURE NAME | PAGE NO. |
|---|---|---|
| 3.1 | EchoGenesis adaptive voice preservation and natural TTS framework (system architecture) | 11 |
| 3.2 | Pipeline for the trained EchoGenesis model | 12 |
| 4.1 | EchoGenesis user interface: voice and language selection with emergency phrases | 23 |
| 4.2 | EchoGenesis user interface: common phrases and free-text input | 24 |
| 5.1 | Drift score over degradation (speech slowed by 0–50%) | 25 |
| 5.2 | Drift score from speaker identity robustness | 25 |
| 5.3 | Embedding drift from a fixed anchor across real recordings | 26 |


## LIST OF TABLES


| TABLE NO | TABLE NAME | PAGE NO. |
|---|---|---|
| 2.1 | Summary of related work | 7 |
| 3.1 | Minimum hardware configuration (development / single user) | 12 |
| 3.2 | Recommended hardware configuration (training / faster synthesis) | 12 |
| 3.3 | Software tools | 13 |
| 4.1 | Study phases, inputs, outputs and gate criteria | 17 |
| 4.2 | Controlled degradation conditions | 18 |
| 4.3 | API endpoints and functionality | 20 |

## ABBREVIATIONS

| Abbreviation | Full Form |
|---|---|
| **AI** | Artificial Intelligence |
| **ML** | Machine Learning |
| **TTS** | Text-to-Speech |
| **ASR** | Automatic Speech Recognition |
| **ALS** | Amyotrophic Lateral Sclerosis |
| **AAC** | Augmentative and Alternative Communication |
| **ECAPA-TDNN** | Emphasized Channel Attention, Propagation and Aggregation in Time Delay Neural Network |
| **G2P** | Grapheme-to-Phoneme |
| **HiFi-GAN** | High-Fidelity Generative Adversarial Network (neural vocoder) |
| **WER** | Word Error Rate |
| **MOS** | Mean Opinion Score |
| **TORGO** | Database of acoustic and articulatory speech from speakers with dysarthria |
| **GE2E** | Generalized End-to-End loss |
| **CTC** | Connectionist Temporal Classification |
| **SCRF** | Segmental Conditional Random Field |
| **API** | Application Programming Interface |
| **HTTP** | Hypertext Transfer Protocol |
| **UI** | User Interface |
| **PWA** | Progressive Web Application |
| **GPU** | Graphics Processing Unit |
| **RAM** | Random Access Memory |

# CHAPTER 1
# INTRODUCTION

## 1.1 OVERVIEW OF THE PROJECT

### Introduction

A person's voice is a vital part of their identity and an essential tool for communication. It tells others who is speaking, how the speaker feels, and it lets people express themselves confidently. Progressive conditions such as Amyotrophic Lateral Sclerosis (ALS), Parkinson's disease and stroke, as well as laryngectomy, dysarthria or cancer treatment, can gradually reduce a person's ability to speak clearly or take the voice away altogether. As the condition advances, fluency, loudness, pronunciation and speech rate change, making communication difficult even when the person can still form words and understand what is said to them.

Existing voice-cloning and text-to-speech (TTS) systems can synthesize customized speech when they are given sufficient, high-quality recordings of a target speaker. However, a voice model built on healthy recordings cannot adapt by itself to a voice that keeps changing, and using later, degraded recordings directly for synthesis makes the output unclear and less similar to the original speaker.

This project introduces **EchoGenesis**, a longitudinal, degradation-robust voice-preservation framework. The earlier, healthy recordings are retained as the identity reference, while later recordings are compared against that reference instead of simply replacing it. The preserved identity is then used by a few-shot TTS model to speak any text typed by the user in the user's own voice.

### Problem Statement

Progressive speech disorders can lead to a loss of two main attributes: the speaker's speech clarity and the understandable, recognizable vocal characteristics. Current voice-cloning approaches can synthesize personalized speech from target recordings; however, a single voice model often fails to account for the gradual changes in speech patterns during the progression of the disorder. The existing approaches have the following flaws:

- **Dependence on healthy speech:** most systems assume clean, healthy and plentiful recordings of the target speaker.
- **Single-stage voice banking:** a voice is banked once before symptoms begin and is not updated or compared as the speech changes.
- **Identity loss:** using later degraded samples directly for cloning copies the degradation into the synthetic voice and reduces speaker similarity.
- **Poor handling of long text:** long sentences synthesized in one pass often sound robotic, broken or unnatural.

The core problem is therefore the lack of a degradation-strong, longitudinal voice-preservation model that can utilize partially degraded speech recordings, retain the speaker's original

identity from prior recordings, and produce clear, personalized speech from text throughout the progression of the speech disorder.

**Objectives of the Project**

The main objectives of this project are:

- To design and develop EchoGenesis, an AI-enabled voice-preservation system for individuals at risk of losing their voices.
- To allow individuals to preserve their voice identity using fragmented or decaying speech samples.
- To reproduce high-quality, natural-sounding versions of the user's voice from typed text.
- To adapt to the user's changing vocal patterns due to conditions such as ALS, laryngectomy or dysarthria.
- To enable users to maintain their voice identity and communicate confidently despite their illness.

**System Overview**

The system integrates the following major components:

1. **Data & Input:** typed or AAC text is cleaned, normalized and converted into phonetic and linguistic embeddings (G2P).
2. **Dynamic Voice Bank and Identity Modeling:** a longitudinal voice repository (T0 → T1 → T2 → Tn, healthy to degraded) feeds an ECAPA-TDNN speaker encoder and a condition encoder whose outputs are combined by identity fusion into a preserved voice identity.
3. **Few-Shot TTS:** a transformer-based acoustic model, conditioned on the preserved identity and prosody, produces a Mel-spectrogram that a neural vocoder (HiFi-GAN) converts into the final waveform.
4. **Evaluation:** speaker similarity, word error rate, mean opinion score and a degradation-robustness curve.

**Scope of the Project**

The scope defines the functionalities, technical boundaries and applications of the system.

**A. Functional Scope**

- **Voice Input:** microphone recordings captured in the browser (check-ins) and typed text.
- **Identity Tracking:** every check-in is scored against the healthy baseline (similarity and drift).
- **Speech Generation:** text is converted to speech in a male or female voice profile, with eight emotion styles.

- **Quick Phrases:** twenty emergency phrases and fifteen common phrases are available as one-tap buttons.
- **Caregiver View:** a simple page listing check-ins and voice-identity match.

**B. Technical Scope**

- **Frontend:** HTML, CSS and JavaScript progressive web application (service worker and web manifest included).
- **Backend:** Python FastAPI server (app.py) that exposes the speaking, phrase-bank and voice-listing API.
- **Speaker Encoder:** SpeechBrain ECAPA-TDNN (pretrained), PyTorch.
- **Speech Synthesis Model:** Coqui XTTS-v2 (Coqui TTS, PyTorch), fine-tuned on the team's recorded voice data.
- **Storage:** file-based voice bank (JSON manifest, PyTorch embedding files, WAV), file-based phrase bank and SQLite speech cache for the prototype; PostgreSQL and blob storage proposed for deployment.

**C. Application Scope**

- Home care for people with ALS, Parkinson's disease, stroke or dysarthria.
- Pre- and post-operative support for laryngectomy and throat cancer patients.
- Speech-language therapy clinics and rehabilitation centres.

**Importance of the Project**

A voice is more than a communication channel; it is a part of a person's identity. Losing it affects independence, relationships and emotional well-being. The key benefits of this project include:

- **Independence:** Independence: a person can keep communicating in their own voice rather than with a generic synthetic voice.
- **Identity and relationships:** Identity and relationships: a familiar voice helps family and friends recognise the speaker in everyday conversation.
- **Emotional well-being:** Emotional well-being: keeping a voice that sounds like one's own may ease the distress of losing speech, although this is not yet measured.
- **Clinical follow-up:** Clinical follow-up: the longitudinal drift record gives clinicians a measurable view of how the voice is changing over time.
- **Immediate communication:** Immediate communication: quick emergency and common phrases can be spoken with one tap, using real recordings for critical messages.

**Challenges and Limitations**

- Embedding drift varies from session to session because of recording conditions and the speaker's state on a given day.

- Accuracy depends on the quality and diversity of the training data (the TORGO corpus has a small number of speakers).
- Severely degraded recordings carry less speaker information and make few-shot adaptation difficult.
- Real-time synthesis on CPU-only machines can be slow for long text.
- Ethical issues such as consent, privacy and misuse of voice cloning must be addressed for real deployment.

# CHAPTER 2
# LITERATURE REVIEW

## 2.1 LITERATURE OVERVIEW

Speech synthesis has moved from rule-based and concatenative methods to neural architectures that can imitate the voice of a particular speaker from only a few seconds of audio. At the same time, assistive technology for people with speech impairments has progressed from fixed synthetic voices to personalized “voice banking”, in which a person’s voice is recorded and reconstructed before it is lost. Most of this work, however, assumes healthy, clean recordings and treats voice banking as a one-time activity.

This literature review focuses on speaker-conditioned speech synthesis, voice preservation for progressive speech disorders, voice cloning of dysarthric speech, speech recognition and enhancement, and augmentative and alternative communication (AAC). The goal is to identify the research gap that EchoGenesis aims to fill.

This review serves multiple purposes:

- To identify the research gap in existing voice-preservation techniques.
- To understand the role of speaker encoders, few-shot TTS models and neural vocoders in a deployable system.
- To understand how dysarthric speech corpora such as TORGO are used for evaluation.
- To provide the theoretical foundation and justification for the proposed system.

## 2.2 LITERATURE SURVEY

The literature survey is organized into thematic sub-sections to provide a structured understanding of the related research, from speaker-conditioned synthesis to assistive communication.

### A. Speech Synthesis

The field of speech synthesis has seen the development of speaker-conditioned architectures that give a speaker representation as input to a text-to-speech system. Jia et al. proposed a transfer-learning approach that separates a speaker’s representation from the steps of speech synthesis and waveform generation. It uses a speaker encoder trained with an end-to-end loss [10], a synthesizer built on Tacotron 2 [11] and a WaveNet vocoder [4]. This separation gave a basis for adapting a general speech-synthesis system to a target speaker from speaker-embedding information only, an approach carried forward in few-shot voice-cloning methods that learn a new speaker from only a handful of reference samples [12], in speaker-conditioned TTS work on Indian and Western accents [1], and in an alternative multi-speaker transfer-learning formulation [5].

### B. Voice Preservation for Progressive and Declining Speech Disorders

A smaller body of work focuses on preserving the voice of people with progressive or declining voice conditions. Veaux et al. showed that synthetic voices can be constructed from a set of healthy speech recordings and later adapted to the effects of the disorder [13]. More recently, Regondi et al. investigated AI-generated, patient-specific voices for people with ALS by fine-tuning a HiFi-GAN-based synthesis pipeline on banked recordings, with results close to the original recordings [14]. The literature therefore shows that voice preservation is a usable assistive technology for degenerative speech disorders, but it typically requires only one round of voice banking before symptoms appear.

### C. Voice Cloning and Dysarthric Speech

Recent works have explored voice cloning of dysarthric speech under few-shot settings, mostly based on the TORGO database of dysarthric and control speech [15]. Moell and Aronsson investigated voice cloning of dysarthric speech on TORGO and revealed limitations caused by the small number of samples and unnatural or linguistically incorrect output [3]. Fairness-oriented studies of F5-TTS on dysarthric and healthy speakers from TORGO have examined accessibility, speaker similarity, prosody and fairness across severity levels [2]. These motivate the focus of EchoGenesis on separating the effects of speech damage from stable speaker characteristics.

### D. Voice Cloning and Speech Recognition

Zero-shot voice cloning has also been used to create speech that resembles dysarthric speech. Singh et al. evaluated zero-shot voice cloning as a low-effort data augmentation technique for automatic speech recognition and assessed the results using word error rate and cross-corpus generalization [6]. This supports the idea that personalized synthetic speech is useful when target data is scarce, although performance depends strongly on the severity of impairment.

### E. Speech Recognition and Enhancement

Automatic speech recognition (ASR) models can be used to assess whether synthesized speech is intelligible. Radford et al. introduced a large-scale, weakly supervised speech-recognition approach trained on a multilingual audio-transcript collection [8], and related CTC and segmental CRF work provides additional background [9]. EchoGenesis can use an ASR component to convert generated speech back to text and compute the word error rate. Speech enhancement, such as flow-matching-based enhancement [7], is also relevant because the quality of affected recordings influences the speaker representation.

### F. Augmentative and Alternative Communication Technology

Apart from voice cloning, the text and AAC input stage of EchoGenesis is closely related to augmentative and alternative communication. This literature motivates treating typed text and AAC input as first-class entry points into EchoGenesis, alongside recorded speech.

## Summary Table of Related Work

Table 2.1 summarizes the key studies reviewed, with their authors, methodology, datasets, limitations and the gap each addresses.

**Table 2.1 – Summary of related work**

| TITLE | AUTHOR | METHODOLOGY | DATA SET | LIMITATION | ADDRESSES |
|---|---|---|---|---|---|
| [1] Empowering Communication: Speech Technology for Indian and Western Accents through AI-powered Speech Synthesis | Vinotha R, Hepsiba D, L.D. Vijay Anand, Deepak John Reji. 2024. arXiv:2401.11771v2 | Data collection, pre-processing, speaker encoder using LSTM (long short-term memory), text-to-mel synthesizer, WaveRNN vocoder, noise reduction and evaluation. | LibriSpeech, NPTEL2020 Indian English, LJSpeech, VCTK and AccentDB datasets. | The system still finds it difficult to accurately copy complex emotions and expressions in natural human speech. | Voice cloning struggles to recreate subtle emotional cues, leaving synthesized speech feeling unnaturally flat. |
| [2] Fairness in Dysarthric Speech Synthesis: Understanding Intrinsic Bias in Dysarthric Speech Cloning using F5-TTS | Anuprabha M, Krishna Gurugubelli, Anil Kumar Vuppala. 15 August 2025. arXiv:2508.05102v3 | F5-TTS clones dysarthric speech from TORGO and evaluates it for intelligibility, speaker similarity, prosody and fairness. | TORGO dataset with 8 dysarthric and 7 healthy speakers, covering low, medium and high dysarthria severity. | Synthetic speech can be harder to understand in severe dysarthria, and too much synthetic data may reduce recognition accuracy. | Separating vocal decline from speaker traits allows intelligible, customized voice retention across changing stages. |
| [3] Voice Cloning for Dysarthric Speech Synthesis: Addressing Data Scarcity in Speech-Language Pathology | Birger Moell, Fredrik Sand Aronsson. 3 March 2025. arXiv:2503.01266v1 | Data collection, TORGO dataset, audio selection, ElevenLabs voice cloning, synthetic speech generation, speech-language pathologist (SLP) evaluation and performance evaluation. | TORGO dataset, containing speech recordings from speakers with dysarthria (ALS/CP) and healthy controls. | Small number of samples, only English speakers, some samples had only single words, and some artificial speech had language mistakes that made it sound unnatural. | Disentangling progressive speech impairment from identity enables clear, personal voice preservation over time. |
| [4] Transfer Learning from Speaker Verification to Multispeaker Text-To-Speech Synthesis | Ye Jia, Yu Zhang, Ron J. Weiss, Quan Wang, Jonathan Shen, Fei Ren, Zhifeng Chen, Patrick Nguyen, Ruoming Pang, Ignacio Lopez Moreno, Yonghui Wu. 2 Jan 2019. arXiv:1806.04558v4 | Three-part decoupled system: speaker encoder, Tacotron 2 based synthesizer and WaveNet vocoder. | LibriSpeech, Voice Cloning Toolkit (VCTK), VoxCeleb, VoxCeleb2 and a proprietary voice search corpus. | Reduced speaker similarity on unseen speakers, inability to transfer accents, and fixed embedding capacity limits. | Provides the speaker-encoder plus synthesizer approach on which speaker-conditioned synthesis in EchoGenesis builds. |
| [5] Voice Cloning: A Multi-Speaker Text-to-Speech Synthesis Approach Based on Transfer Learning | Giuseppe Ruggiero, Enrico Zovato, Luigi Di Caro, Vincent Pollet. 10 Feb 2021. arXiv:2102.05630v1 | Three-part decoupled system: speaker encoder (SE), sequence-to-sequence (Seq2Seq) Tacotron 2-based synthesizer and WaveRNN-based vocoder. | LibriTTS, VoxCeleb, VoxCeleb2, CSTR Voice Cloning Toolkit (VCTK) and a proprietary voice corpus. | Reduced speaker similarity (SS) on unseen speakers, inability to capture target speaker prosody (SP), and lower naturalness compared to single-speaker models. | Shows how transfer learning from a speaker encoder gives multi-speaker voice cloning from a small amount of target speech. |
| [6] Low-Burden Data Augmentation for Dysarthric ASR via Zero-Shot Voice Cloning | Satwinder Singh, Qianli Wang, Zihan Zhong, Clarion Mendes, Mark Hasegawa-Johnson, Waleed Abdulla, Seyed Reza Shahamiri. 18 June 2026. arXiv:2606.19823v1 | Zero-shot voice cloning, synthetic speech generation, Whisper fine-tuning and WER-based evaluation. | TORGO dataset: approximately 23 hours of speech from 8 dysarthric speakers (5 male, 3 female), mainly with cerebral palsy (CP) and ALS. | Synthetic data may create unnatural sounds, and its effectiveness can change depending on the severity of dysarthria. | Using zero-shot voice cloning for low-burden data augmentation improves dysarthric ASR performance and cross-corpus generalization. |

| TITLE | AUTHOR | METHODOLOGY | DATA SET | LIMITATION | ADDRESSES |
|---|---|---|---|---|---|
| [7] FlowSE: Flow Matching-based Speech Enhancement | Seonggyu Lee, Sangwook Han, Sein Cheong, Jong Won Shin. 9 Aug 2025. arXiv:2508.06840v1 | Flow matching-based generative approach to speech enhancement. | LibriTTS, VoxCeleb and a proprietary voice corpus. | Reduced speaker similarity (SS) on unseen speakers, inability to capture target speaker prosody (SP). | Cleaner input speech improves the quality of the speaker representation extracted from degraded recordings. |
| [8] Robust Speech Recognition via Large-Scale Weak Supervision | Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, Ilya Sutskever. 6 Dec 2022. arXiv:2212.04356v1 | Uses large-scale multilingual audio data, filters transcripts, converts audio into mel-spectrograms and trains an encoder-decoder Transformer model. | 680,000-hour web-scale audio-transcript dataset. | It may make mistakes when transcribing long speech and needs a lot of computing power. | Provides accurate speech recognition in different languages and environments without needing specific training for each one. |
| [9] Multitask Learning with CTC and Segmental CRF for Speech Recognition | Liang Lu, Lingpeng Kong, Chris Dyer, Noah A. Smith. 5 Jun 2017. arXiv:1702.06378v4 | Jointly trains CTC (connectionist temporal classification) and segmental CRF (conditional random field) using a shared LSTM-RNN encoder with an interpolated loss. | TIMIT speech recognition dataset; the experiments follow the standard protocol using the core test set of 192 utterances. | SCRF needs a lot of computing power, and its context-based embedding may slightly reduce recognition accuracy. | Improves speech recognition accuracy using CTC and SCRF, while making training faster through CTC pretraining. |

**Research Gaps Identified**

- **Single-stage Voice Banking:** existing work banks a voice once, before symptoms, and does not track how the voice changes afterwards.
- **Identity versus Degradation:** few systems separate the stable speaker identity from the effects of speech damage.
- **Limited Integration:** many studies present a model with good metrics but not a complete pipeline with a usable interface for patients and caregivers.
- **Long Utterances:** personalized TTS often sounds broken on long sentences; prosody-aware chunking is rarely addressed.

This project directly addresses these gaps through a longitudinal voice bank, a degradation-robust speaker encoder, few-shot TTS with prosody-aware chunked synthesis and an end-to-end web application.

Each research gap above corresponds to a component of EchoGenesis. Single-stage voice banking is addressed by the longitudinal voice bank, which stores every session and compares it with the baseline (Section 4.2.3). The identity-versus-degradation gap is addressed by the degradation-robust speaker encoder and the fixed baseline reference (Section 4.2.2). Limited integration is addressed by the FastAPI backend and the progressive web interface (Section 4.2.5). Long utterances are addressed by prosody-aware chunked synthesis (Section 4.1 F).

**Summary of Literature Findings**

The reviewed literature demonstrates the evolution of speech technology from healthy-speech, single-speaker systems to few-shot and zero-shot voice cloning, and from one-time voice banking to dysarthric speech synthesis. While advances in speaker encoders and neural vocoders enable accurate cloning of healthy voices, a significant gap remains: there is a clear lack of integrated systems that preserve speaker identity continuously across changing speech conditions. This project aims to bridge that gap with a complete, end-to-end software solution.

# CHAPTER 3
# SYSTEM DESIGN

## 3.1 EXISTING SYSTEM

### Overview

Existing solutions for people who are losing their speech fall into three groups: (i) generic text-to-speech (TTS) engines with a fixed synthetic voice, (ii) voice-cloning and voice-banking services that build a personalized voice from clean recordings, and (iii) AAC devices that provide a symbol or text interface. Voice banking is normally performed once, before the speech starts to deteriorate, and the resulting model is not updated afterwards.

### Limitations of Existing Systems

- **Healthy-speech Assumption:** voice-cloning systems need sufficient, high-quality recordings of healthy speech.
- **No Adaptation Over Time:** a voice model built once cannot follow gradual changes in the person's voice.
- **Identity Loss:** directly cloning later, degraded recordings reduces intelligibility and speaker similarity.
- **Impersonal Output:** generic TTS voices do not sound like the user and reduce the sense of identity.
- **Long Sentences:** one-pass synthesis of long text often sounds robotic or broken.

The existing systems are therefore insufficient for preserving a person's identity throughout the course of a progressive speech disorder.

## 3.2 PROPOSED SYSTEM

### Overview

The proposed system, **EchoGenesis**, is a longitudinal, degradation-robust voice-preservation framework. The user first records relatively healthy speech, which is stored as the primary reference for the speaker's voice. Later recordings, which may be short or unclear, are accepted as additional sessions of the same speaker rather than as different voices. A degradation-robust speaker encoder extracts identity from every session, and the system compares the latest session with the earliest one. When the user can no longer speak clearly, text typed in the interface is converted to speech in the preserved voice.

### Features of the Proposed System

- **Dynamic Voice Bank:** recordings are stored in chronological order from healthy to degraded.
- **Degradation-Robust Speaker Encoder:** SpeechBrain ECAPA-TDNN with a condition encoder and identity fusion.
- **Few-Shot TTS:** speech is generated from a small number of voice samples.

- **Prosody-Aware Chunked Synthesis:** long text is divided into meaningful segments and joined smoothly.
- **Emotion Control:** neutral, urgent, happy, calm, sad, surprised, playful and whisper styles.
- **Quick Phrases:** emergency and common phrases for one-tap communication.
- **Accessibility:** switch scanning, eye-tracking (experimental) and a high-contrast interface.
- **Caregiver View:** shows voice-identity match over time.

## 3.3 SYSTEM ARCHITECTURE

### Architectural Overview

The EchoGenesis architecture is modular: a data and input block, a dynamic voice bank, identity modeling, a few-shot TTS block and an evaluation block, together with a training and adaptation loop, as shown in Figure 3.1. Each block can be developed, tested and replaced independently.

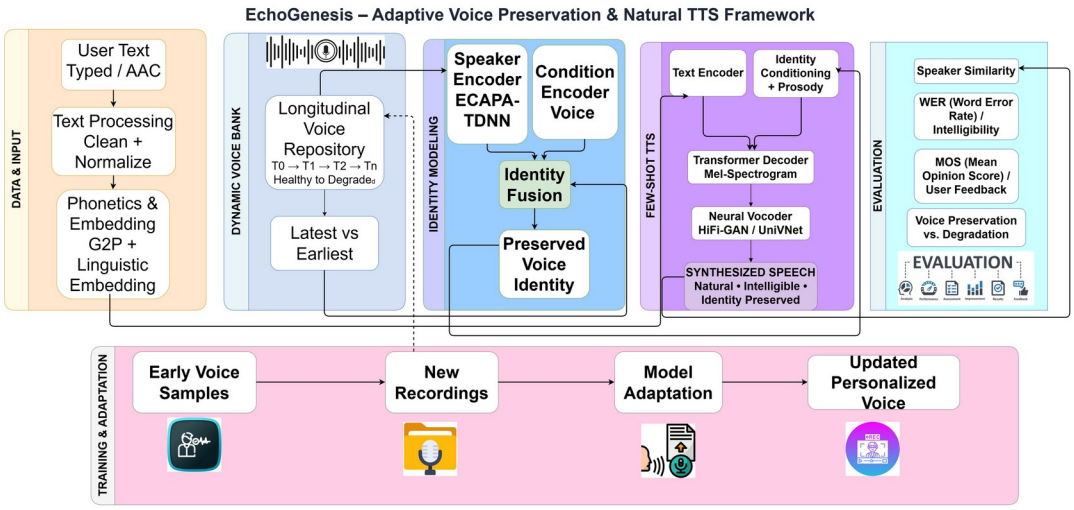



**Figure 3.1 – EchoGenesis adaptive voice preservation and natural TTS framework**

The lower part of Figure 3.1 shows the training and adaptation loop: early voice samples are collected first, new recordings are added over time, the model is adapted and an updated personalized voice is produced.

### Training Pipeline

Figure 3.2 shows the proposed training pipeline. Speech-and-text paired data is cleaned and augmented with noise, reverberation and codec variation before encoder training. The ECAPA-TDNN speaker encoder is fine-tuned for robustness to the augmented conditions, and the SpeechT5-based acoustic model is fine-tuned to disentangle linguistic content from speaking style using the reference speaker's voice. Synthesis uses chunking and a vocoder, followed by WER and MOS evaluation.

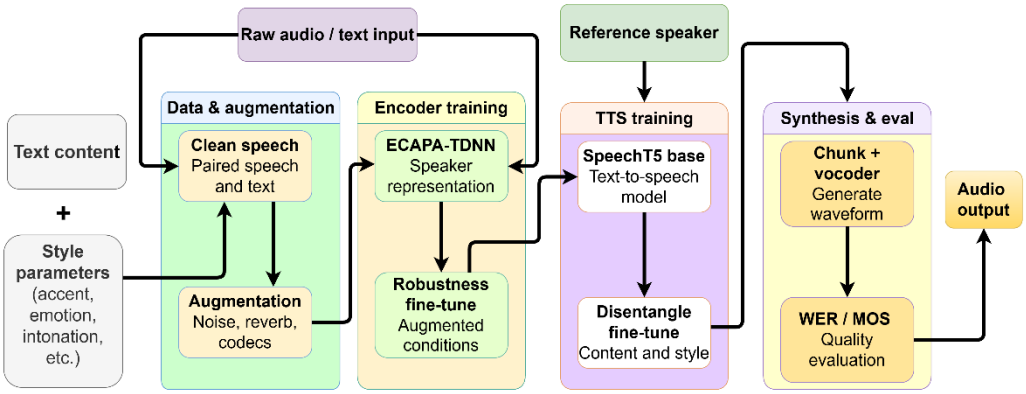



**Figure 3.2 – Pipeline for the trained EchoGenesis model**

**Privacy and Ethics**

- Voice recordings are processed on a local server and are not sent to a public cloud.
- The voice bank is stored under the user's own profile and can be deleted by the user.
- The system is meant to preserve the user's own voice; consent of the speaker is required before any voice is banked.

## 3.4 HARDWARE REQUIREMENTS

**Table 3.1 Minimum Hardware Configuration (Development / Single User)**

| Component | Specification |
|---|---|
| Processor | Intel Core i5 / AMD Ryzen 5 |
| RAM | 8 GB |
| Storage | 256 GB SSD |
| Microphone | Built-in or external microphone |
| Display | Any display with a modern web browser |

**Table 3.2 Recommended Hardware Configuration (Training / Faster Synthesis)**

| Component | Specification |
|---|---|
| Processor | Intel Core i7 / AMD Ryzen 7 or higher |
| RAM | 16 GB or more |
| Storage | 512 GB NVMe SSD |
| GPU | NVIDIA GPU with CUDA support (for fine-tuning and faster inference) |
| Microphone | Good-quality USB microphone for recording the voice bank |

**Hardware Justification:** the TTS model and the vocoder are the main computational load. A CPU is sufficient for a single user and short sentences, whereas fine-tuning the encoder and

acoustic model, or synthesizing long text quickly, benefits from a GPU. A good-quality microphone improves the quality of the stored baseline.

## 3.5 SOFTWARE REQUIREMENTS

**Table 3.3 Software Tools**

| Software | Description |
|---|---|
| Operating System | Windows 10/11, macOS or Ubuntu 20.04 |
| Programming Language | Python 3.10+ |
| PyTorch | Deep learning framework used by the speaker encoder and TTS models |
| SpeechBrain (ECAPA-TDNN) | Pretrained speaker encoder for identity embeddings and similarity |
| Coqui TTS (XTTS-v2) | Multilingual voice-cloning TTS model and trainer, fine-tuned on recorded voice data |
| FastAPI, Uvicorn | Backend web framework and server that exposes the inference API |
| SoundFile, NumPy, Librosa | Reading/writing WAV, resampling, noise reduction, trimming and normalization of reference recordings |
| SQLite (Python standard library) | Speech-cache index and autocomplete history |
| HTML5, CSS3, JavaScript | Frontend web interface, MediaRecorder API, service worker (PWA) |
| jsdom (Node.js) | Frontend smoke tests for offline phrase playback and switch scanning |
| React Native (proposed) | Mobile application for recording, text input and playback |
| PostgreSQL + blob storage (proposed) | Session metadata and embeddings; audio storage for deployment |
| Web Browser | Google Chrome, Firefox, Edge or Safari |

**Software Modules**

- **index.html (frontend):** voice and language selection, emergency and common phrases, emotion chips, text input with suggestions, voice check-in recorder, accessibility panel and audio player.
- **app.py (FastAPI server):** routes /health, /voices, /speak and /phrases, an optional API-key check, and static mounts for recorded phrases (/bank) and cached audio (/cache).
- **config.py:** project paths, voice definitions (checkpoint and reference-clip folders), languages supported by XTTS-v2, number of reference clips (6) and chunk length (200 characters).
- **engine.py:** loads the fine-tuned XTTS-v2 checkpoint (or base XTTS-v2), computes conditioning latents from reference clips, splits text into chunks, synthesizes at 24 kHz and caches the result by hash.

- **phrasebank.py:** stores real recordings per voice and language with a manifest keyed on normalized text; exact-match lookup, add and list.
- **speech_cache.py:** SQLite index of synthesized utterances by text, voice, language, register and emotion, plus autocomplete suggestions from the speaking history.
- **voice_banking.py:** longitudinal voice bank (bootstrap, add session, report) that scores each session against the baseline and ranks reference clips by similarity to it.
- **synthesize_xtts.py, train_xtts.py, regenerate_metadata.py:** XTTS-v2 inference helper, fine-tuning script and training-metadata builder.
- **orchestrate_pipeline_v2.py:** end-to-end command-line pipeline (clean reference recording, chunk text, quick-phrase check, XTTS synthesis, stitching).
- **static/ files:** manifest.json, service worker (sw.js), offline.js, a11y.css and a11y.js for the progressive web app and accessibility features.

# CHAPTER 4
# APPROACH AND EXECUTION

## 4.1 METHODOLOGY

The methodology defines the systematic approach followed in the development of EchoGenesis. It outlines the processes, tools and techniques used to collect voice input, build the speaker identity, synthesize speech and evaluate the results. The methodology ensures that each stage of development is logically structured, reproducible and performance-oriented.

### Research Approach

The research approach is applied and experimental. It combines quantitative evaluation (embedding drift, speaker similarity, word error rate) with qualitative evaluation (naturalness and intelligibility through mean opinion score). The development process involved:

### A. Collection of Input Voice

The initial step in EchoGenesis is the creation of the user's original voiceprint. A set of healthy or clear speech samples is recorded and stored as the primary reference of the speaker's voice, capturing the vocal characteristics before significant decline. Later recordings, which may be short or garbled, are accepted as additional sessions representing different stages of the same speaker rather than as different voices. In addition to speech, the input pipeline accepts typed text or AAC input, which then goes through cleaning, normalization, grapheme-to-phoneme conversion and linguistic embedding.

### B. Preprocessing

Collected audio is preprocessed before model processing: conversion to a common audio format and sampling frequency (mono, 16 kHz), removal of unwanted silences, trimming of speech, normalization and noise reduction where appropriate. This reduces the differences caused by recording conditions and gives the speaker encoder more consistent inputs.

### C. Degradation-Robust Speaker Encoder

A degradation-robust speaker encoder extracts the characteristics that identify the speaker. SpeechBrain with a pretrained ECAPA-TDNN backbone [17] is adopted as the speaker-encoder base. The encoder generates an embedding of the speech signal, and the key requirement is that this representation remains useful even if a later recording is partially degraded. Initial healthy recordings provide the main identity representation, while later recordings add evidence about the same speaker under altered speech conditions. The architecture also includes a condition encoder representing the voice or disease stage and an identity-fusion stage that generates the preserved voice identity.

### D. Longitudinal Comparison

The system compares recent recordings with earlier, healthier ones, not to copy the degradation into the final synthetic voice, but to understand how the current speech representation relates to the stable speaker identity. The former voice is retained for reference and the latter provides

information for adaptation. The voice bank is organized so that recordings are sorted from the earliest healthy samples to later, more damaged ones, which supports progress comparison from the latest recording back to the earliest reference. This comparison is the longitudinal component of EchoGenesis and the major difference between the proposed workflow and a static voice-cloning pipeline.

**E. Few-Shot Text-to-Speech System**

After obtaining the speaker representation, the few-shot TTS model takes the user text together with reference clips of the preserved voice and outputs the corresponding speech. FastSpeech2 [18] and SpeechT5 [19] were considered as acoustic models. In head-to-head testing, SpeechT5 fine-tuning did not give usable results, while XTTS-v2 [21] captured the speaker's real identity (correct gender and timbre) out of the box, so the prototype uses XTTS-v2. Its GPT component is fine-tuned on the team's own recordings (train_xtts.py), and the fine-tuned checkpoint is used whenever it is present, with base XTTS-v2 as the fallback.

**F. Prosody-Aware Chunked Synthesis**

Long sentences are challenging to synthesize smoothly in one pass. EchoGenesis therefore uses prosody-aware chunking, in which a long text input is split at sentence and clause boundaries into smaller meaningful segments. Each segment is synthesized individually while preserving speaker conditioning and prosodic continuity, and the segments are then joined with a short pause into the final audio. This reduces robotic, broken or unnatural transitions and improves clarity for longer messages.

**G. Final Output with a Vocoder**

The acoustic output of the TTS model is converted into an audio waveform. In the prototype, XTTS-v2 produces the waveform directly at 24 kHz through its built-in HiFi-GAN-based decoder [20], so no separate vocoder stage is needed. The resulting waveform is the final synthetic voice that can be played back to communicate. The backend provides inference as an API through FastAPI, and the frontend records audio, accepts text and plays the speech.

**4.1.2 Evaluation Metrics**

- **Speaker Similarity:** embeddings are extracted from reference and generated speech and a similarity is computed between them (generalized end-to-end loss based encoders [10]).
- **Word Error Rate (WER):** generated speech is passed to an ASR system and the hypothesis transcript is compared with the reference transcript; a lower value means clearer speech.
- **Mean Opinion Score (MOS):** human ratings of naturalness and intelligibility.
- **Degradation Robustness Curve:** a metric is computed at different degradation levels and plotted against the distortion.
- **Embedding Drift Score:** the distance in the speaker-encoder embedding space between a candidate recording and an anchor embedding from the speaker's original

healthy recording; the larger the drift, the more the candidate differs from the reference identity.

### H. Study Phases and Workflow

The study runs in five phases, each with a defined output and a gate that must pass before the next phase starts (Table 4.1). Phases 3 and 4 repeat for as long as check-ins continue; the baseline is never overwritten, so drift values recorded months apart stay comparable.

**Table 4.1 – Study phases, inputs, outputs and gate criteria**

| Phase | Activity | Output | Gate criterion |
|---|---|---|---|
| 1. Consent | Written consent and device check | Enrolled speaker profile | Consent recorded before audio is banked |
| 2. Baseline | Record clear, pre-symptom speech | Baseline embedding | Clips pass preprocessing |
| 3. Check-ins | Periodic recordings scored against baseline | Similarity and drift report | Notable drift is flagged for review |
| 4. Synthesis | Reference ranking and generation from text | Speech in preserved voice | Phrase-bank and length checks pass |
| 5. Evaluation | Objective and perceptual tests | Metric tables and curves | Protocol in Sections K and L |

### I. Participant and Recording Protocol

Three speaker groups are used: the primary user (baseline and check-ins, under written consent); dysarthric speakers from the TORGO corpus [15], which test whether identity survives impairment; and healthy TORGO controls, the reference population for speaker similarity. TORGO results are reported per severity level, and no speaker contributes to both training and testing of the same condition.

Every session uses the same procedure: a quiet room, a fixed microphone distance, and a fixed script of sentences and common phrases recorded as separate clips. Each clip is labelled with its session, date and baseline status, so differences between sessions can be attributed to the speaker rather than the setup.

### J. Controlled Degradation Protocol

Real degradation cannot be controlled, so each condition is generated from the same clean clip and changes one property at a time (Table 4.2). Seeds and parameters are stored with each output, so every degraded file can be regenerated exactly.

**Table 4.2 – Controlled degradation conditions**

| Condition | Manipulation | Property tested | Reported as |
|---|---|---|---|
| Clean reference | None | Identity anchor | Zero-drift point |
| Slowdown series | Time-stretched from 0% to 50% slower | Speech rate and timing | Drift against severity |
| Slurred speech | Artificially slurred utterance | Articulation at similar rate | Compared with slowdown |
| Noisy speech | Additive background noise | Noise without timing change | Compared with timing conditions |

### K. Evaluation Procedure

The objective layer computes speaker similarity, word error rate (transcripts from an ASR model, following Radford et al. [8]) and embedding drift on every condition and held-out sentence. The perceptual layer collects mean opinion scores from speech-language pathology practitioners and members of the public, who rate naturalness and intelligibility on a five-point scale. Clips are mixed in random order and listeners do not know which are synthesized.

Each measurement is repeated across sessions and reported with its spread. A change in drift counts as meaningful only when it exceeds the session-to-session spread for the same speaker, a stricter rule than comparing single values.

### L. Reproducibility and Tooling

Random seeds, data splits, model versions, checkpoint locations and run parameters are recorded in the manifest files of the voice bank, so every stage can be repeated. Commands for each stage are listed in Section 4.3.

## 4.2 IMPLEMENTATION

The implementation phase realizes the proposed design as a working system. The system is divided into independent but interrelated modules, each responsible for a specific function. The implementation focuses on modularity, usability and reusability, using the ECAPA-TDNN encoder and the fine-tuned XTTS-v2 model as the core of the pipeline.

### 4.2.1 Voice Input & Preprocessing Module

This module collects the user's voice and text and prepares them for analysis.

**Technology Used:** MediaRecorder API (browser), Librosa, SoundFile, NumPy

**Implementation Steps:**

1. **Recording:** the check-in panel in the interface starts and stops the microphone with the MediaRecorder API and previews the recording before submission.
2. **Storage:** each recording is copied into the voice bank as a WAV file under voice_bank/<voice>/<session>/ (voice_banking.py add <voice> <folder> <label>).

3. **Cleaning:** prepare_reference_audio() loads the recording, converts it to mono 16 kHz and applies noise reduction, trimming and normalization (clean_recording) before it is used as a reference voice.
4. **Text Handling:** typed text is trimmed and normalized (case-folded, punctuation removed) for exact phrase-bank lookup, and split into chunks of up to 200 characters for synthesis.

#### 4.2.2 Speaker Encoder & Identity Modeling Module

This module converts audio into speaker embeddings that represent identity.

**Technology Used:** PyTorch, ECAPA-TDNN speaker embeddings (extract_embedding), XTTS-v2 conditioning latents

**Implementation Steps:**

1. **Embedding:** extract_embedding() computes a speaker embedding for every clip, saved next to the clip as an _embedding.pt file.
2. **Baseline:** the embeddings of the baseline-session clips (bootstrapped from the earliest, healthiest recordings) are averaged into the preserved identity reference.
3. **Reference Selection:** weighted_references() ranks all stored clips by cosine similarity to the baseline and returns the top ones, so synthesis favors the original healthy voice rather than the latest recording.
4. **Conditioning Latents:** engine.py computes the XTTS-v2 conditioning latents (GPT latent and speaker embedding) from up to six reference WAV clips of the chosen voice and caches them per voice.

#### 4.2.3 Longitudinal Voice Bank Module

This module stores every session and tracks how the voice changes over time.

**Technology Used:** JSON manifest, PyTorch tensors, cosine similarity

**Implementation Steps:**

1. **Session Storage:** each clip is copied to voice_bank/<voice>/<session>/ with its embedding file, and manifest.json records the session, WAV path, embedding path, date and whether the clip belongs to the baseline.
2. **Similarity:** the cosine similarity between each non-baseline clip embedding and the baseline mean is computed and averaged per session (avg_similarity).
3. **Drift Report:** drift_report() returns the sessions in date order with their average similarity; the command-line report marks a similarity below 0.75 as notable drift.
4. **Command-line Use:** python voice_banking.py bootstrap <voice>, add <voice> <folder> <label> and report <voice>.

#### 4.2.4 Few-Shot TTS & Chunked Synthesis Module

This module converts text to speech in the preserved voice.

**Technology Used:** Coqui TTS (XTTS-v2), PyTorch, SoundFile, NumPy

**Implementation Steps:**

1. **Fine-tuning:** train_xtts.py fine-tunes the XTTS-v2 GPT component from the base checkpoint on the corrected, denoised English dataset (95 clips: 80 scripted sentences and 15 common phrases) with batch size 1, gradient accumulation of 16, a learning rate of 5e-6, up to 30 epochs and a maximum audio length of 11 seconds; regenerate_metadata.py builds the training and evaluation metadata files.
2. **Chunking:** engine.py splits the text at sentence boundaries into chunks of up to 200 characters; longer sentences are cut at spaces.
3. **Generation:** the fine-tuned checkpoint is loaded if best_model.pth, config.json and vocab.json exist (otherwise base XTTS-v2), and every chunk is synthesized with the cached conditioning latents at a temperature of 0.7, giving 24 kHz audio.
4. **Joining and Caching:** chunks are joined with 150 ms gaps and saved under a SHA-1 hash of voice, language and text, so a repeated sentence is served instantly; speech_cache.py adds an SQLite index and autocomplete suggestions.

### 4.2.5 Web Interface & API Module

This module connects the user with the models.

**Technology Used:** HTML5, CSS3, JavaScript, FastAPI, Uvicorn, Pydantic

**Table 4.3 API Endpoints and Functionality**

| Endpoint | Method | Purpose |
|---|---|---|
| /health | GET | Health check |
| /voices | GET | Lists the voice profiles and the languages XTTS-v2 can synthesize |
| /speak | POST | Takes text, voice, language and an optional source language (translated first if different). Returns a recorded phrase from the phrase bank if the text matches exactly, otherwise a synthesized audio URL; unsupported languages return no audio so the client can use on-device TTS |
| /phrases | GET | Lists the recorded phrases of a voice and language with their audio URLs |
| /phrases | POST | Uploads a recording and its text to the phrase bank (a re-recording overwrites the old one) |
| /bank/, */cache/* | GET | Static audio of recorded phrases and cached synthesized speech |

Every /speak request is first looked up in the phrase bank (phrasebank.py). Matching uses the normalized text (Unicode NFKC, case-folded, punctuation removed, spaces collapsed) and only exact matches count, so emergency phrases are always spoken from the real recording and never approximated by the model. The API key is read from the ECHO_API_KEY

environment variable and checked through the X-API-Key header when it is set. A jsdom-based smoke test (smoke_frontend.js) checks offline phrase playback, press logging and switch scanning of the web interface.

The interface has a splash screen, male and female voice toggle, language toggle (English and Tamil enabled; Hindi and Malayalam reserved), Tamil casual and formal register, emergency and common phrase grids, eight emotion chips, a text box with live suggestions, a Speak button and an audio player. An accessibility panel provides switch scanning with adjustable speed and experimental eye-tracking with dwell selection. A service worker and web manifest allow the application to be installed and to work offline for cached content.

## 4.3 SOURCE CODE

Key excerpts of the source code are given below: the project folders, the commands used to run it, the FastAPI /speak route (app.py), the chunked synthesis routine (engine.py) and the longitudinal drift report (voice_banking.py). The remaining modules are described in Section 3.5 and are not reproduced here to keep the report within the page limit. At run time the project uses the following folders (their paths are set in config.py and the scripts):

```
phrase_bank/<voice>/<language>/   recorded phrases and manifest.json
cache/                            synthesized speech (hash-named WAV files)
finetuned_model/<voice>/          best_model.pth, config.json, vocab.json
finetune_dataset/<voice>/wavs/    recordings, metadata_train.csv, metadata_eval.csv
voice_bank/<voice>/               baseline and later sessions, embeddings, manifest.json
xtts_training_output/             training runs and checkpoints
final_app/                        speech_cache.db, cache_audio/, static/ (web interface)
```

**Running the project:**

```
cd backend
uvicorn app:app --host 127.0.0.1 --port 8000
# optional: set ECHO_ROOT (project root) and ECHO_API_KEY before starting

python train_xtts.py                      # fine-tune XTTS-v2
python voice_banking.py bootstrap male    # seed the baseline session
python voice_banking.py add male <folder> <label>
python voice_banking.py report male
node tests/smoke_frontend.js              # needs: npm install jsdom
```

**app.py (Backend API – Python / FastAPI)**

```
from typing import Optional
from fastapi import FastAPI, Header, HTTPException, UploadFile, Form, File, Depends
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel
import config, phrasebank
from translate import translate

config.PHRASE_BANK.mkdir(parents=True, exist_ok=True)
config.CACHE_DIR.mkdir(parents=True, exist_ok=True)

app = FastAPI(title="Echogenesis")
app.mount("/bank", StaticFiles(directory=config.PHRASE_BANK), name="bank")
app.mount("/cache", StaticFiles(directory=config.CACHE_DIR), name="cache")  # TODO: protect in
production


def auth(x_api_key: str = Header(default="")):
    if config.API_KEY and x_api_key != config.API_KEY:
        raise HTTPException(401, "bad api key")


class SpeakReq(BaseModel):
```

```
    text: str
    voice: str = "male"
    language: str = "en"                # language to speak in
    source_language: Optional[str] = None  # if set and different, text is translated first


@app.get("/health")
def health():
    return {"ok": True}


@app.get("/voices", dependencies=[Depends(auth)])
def voices():
    return {"voices": list(config.VOICES), "synth_languages": sorted(config.XTTS_LANGS)}


@app.post("/speak", dependencies=[Depends(auth)])
def speak(r: SpeakReq):
    if r.voice not in config.VOICES:
        raise HTTPException(400, "unknown voice")
    text = r.text.strip()
    if not text:
        raise HTTPException(400, "empty text")
    if r.source_language and r.source_language != r.language:
        text = translate(text, r.source_language, r.language)

    hit = phrasebank.lookup(r.voice, r.language, text)
    if hit:
        return {"source": "recorded", "text": text, "audio_url":
f"/bank/{r.voice}/{r.language}/{hit.name}"}

    if r.language not in config.XTTS_LANGS:
        # client should fall back to on-device TTS for this language
        return {"source": "unsupported", "text": text, "audio_url": None}

    import engine
    wav = engine.synthesize(r.voice, r.language, text)
    return {"source": "synthesized", "text": text, "audio_url": f"/cache/{wav.name}"}
```

**engine.py (XTTS-v2 synthesis engine with caching)**

```
def _split(text, limit=MAX_CHUNK_CHARS):
    parts = re.split(r"(?<=[.!?。 ! ? ])\s+", text.strip())
    out, cur = [], ""
    for p in parts:
        if len(cur) + len(p) + 1 <= limit:
            cur = f"{cur} {p}".strip()
        else:
            if cur: out.append(cur)
            while len(p) > limit:
                cut = p.rfind(" ", 0, limit); cut = cut if cut > 0 else limit
                out.append(p[:cut]); p = p[cut:].strip()
            cur = p
    if cur: out.append(cur)
    return out


def synthesize(voice, lang, text) -> Path:
    """Return path to a wav for `text`. Results are cached, so repeats are instant."""
    CACHE_DIR.mkdir(parents=True, exist_ok=True)
    out = CACHE_DIR / (hashlib.sha1(f"{voice}|{lang}|{text}".encode()).hexdigest() + ".wav")
    if out.exists():
        return out
    with _lock:
        key, model = _load_model(voice)
        if voice not in _latents:
            _latents[voice] = model.get_conditioning_latents(audio_path=_clips(voice))
        gpt, spk = _latents[voice]
        pieces, gap = [], np.zeros(int(SR * 0.15), dtype=np.float32)
        for chunk in _split(text):
            r = model.inference(chunk, lang, gpt, spk, temperature=0.7)
            pieces += [np.asarray(r["wav"], dtype=np.float32), gap]
        sf.write(out, np.concatenate(pieces[:-1]), SR)
    return out
```

**voice_banking.py (Longitudinal voice bank)**

```
def _baseline_embedding(voice):
    manifest = _load_manifest(voice)
    baseline_embs = [
        torch.load(c["embedding_path"], weights_only=False)
        for c in manifest["clips"] if c["is_baseline"]
    ]
    if not baseline_embs:
        return None
    return torch.stack(baseline_embs).mean(dim=0)


def _similarity(a, b):
    return torch.nn.functional.cosine_similarity(a.unsqueeze(0), b.unsqueeze(0)).item()


def drift_report(voice):
    """Returns a list of {session, date, avg_similarity} for every
    non-baseline session, sorted chronologically."""
    baseline_emb = _baseline_embedding(voice)
    if baseline_emb is None:
        return []

    manifest = _load_manifest(voice)
    sessions = {}
    for c in manifest["clips"]:
        if c["is_baseline"]:
            continue
        emb = torch.load(c["embedding_path"], weights_only=False)
        sim = _similarity(emb, baseline_emb)
        sessions.setdefault(c["session"], {"date": c["date"], "sims": []})
        sessions[c["session"]]["sims"].append(sim)

    report = []
    for session, info in sessions.items():
        avg_sim = sum(info["sims"]) / len(info["sims"])
        report.append({"session": session, "date": info["date"], "avg_similarity": round(avg_sim, 4)})

    return sorted(report, key=lambda r: r["date"])
```

### 4.3.1 Web Application Output

Figures 4.1 and 4.2 show the working EchoGenesis interface. The user can choose the male or female voice and the language, and then speak with one tap using the emergency phrases, the common phrases or free text in the text box.

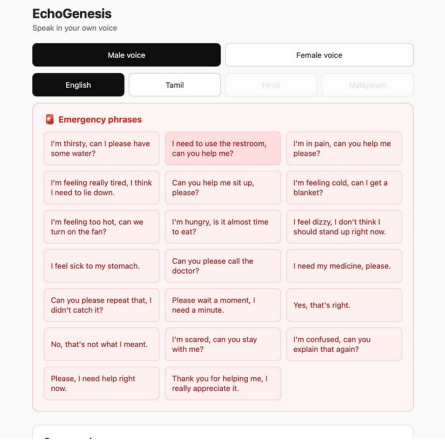



**Figure 4.1– User interface: voice and language selection with emergency phrases**

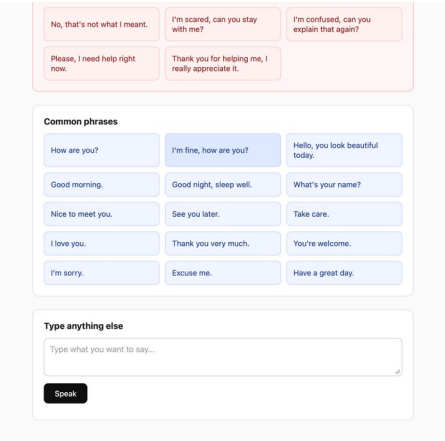



**Figure 4.2– User interface: common phrases and free-text input (Speak)**

# CHAPTER 5
# RESULT ANALYSIS

## A. Proof-of-Concept Embedding Drift Study

Before the full TORGO-based evaluation, a lightweight demonstration was run in a Google Colab notebook (ECHOGENESIS.ipynb) to check whether an embedding-based drift score is meaningful under controlled degradation. The drift score is the distance, in the speaker-encoder embedding space, between a candidate recording and an anchor embedding from the speaker's original healthy recording.

Figure 5.1 plots the drift score against the severity of degradation, measured as the amount of slowdown relative to the original recording. Drift increased monotonically with severity, from 0 to roughly 0.22 at 50% slowdown, with the greatest increase between 0% and 10%. This shows the expected monotonic relationship: the more degraded the audio, the farther it drifts from the original identity embedding.

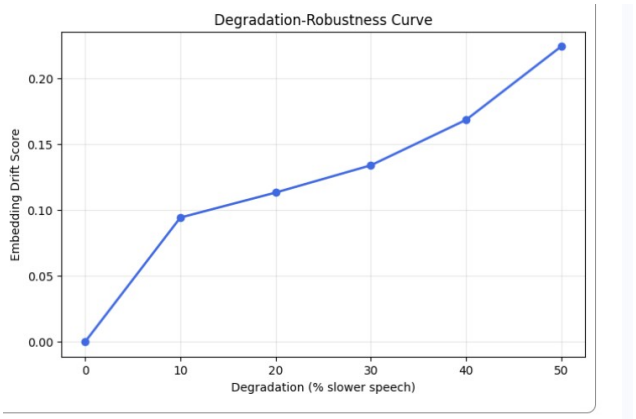



**Figure 5.1 – Embedding drift score over degradation (speech slowed by 0–50%)**

A second experiment compared drift scores between synthetic degraded versions of the same speaker: audio slowed by 20%, an artificially slurred version and a noisy version, against a clean baseline fixed at zero drift (Figure 5.2). The slowed and slurred conditions produced similar, relatively high drift (approximately 0.31 and 0.30), while the noisy condition produced a much smaller drift (approximately 0.10). Different degradation types therefore affect the embedding space unevenly.

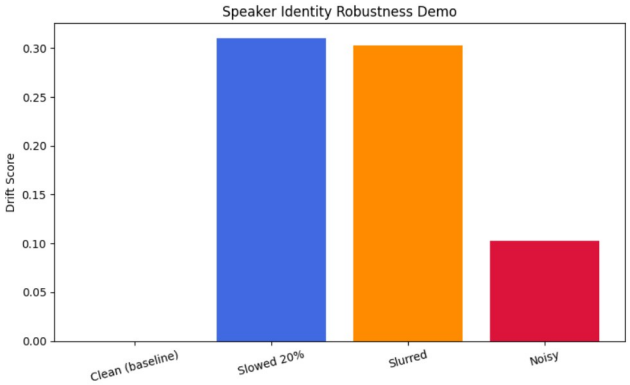



**Figure 5.2 – Drift score from speaker identity robustness**

Beyond synthetic degradation, drift was also measured for one real speaker across recording sessions relative to a single anchor embedding from an earlier session (Figure 5.3). Drift increased from 0.155 in session 2 to a peak of 0.555 in session 3, dropped back to 0.273 in session 4 and rose again to 0.328 in session 5. This profile is not monotonic, which shows that session-level variability (for example different recording conditions or the speaker's state on a given day) can be larger than the gradual voice change that the system is trying to detect.

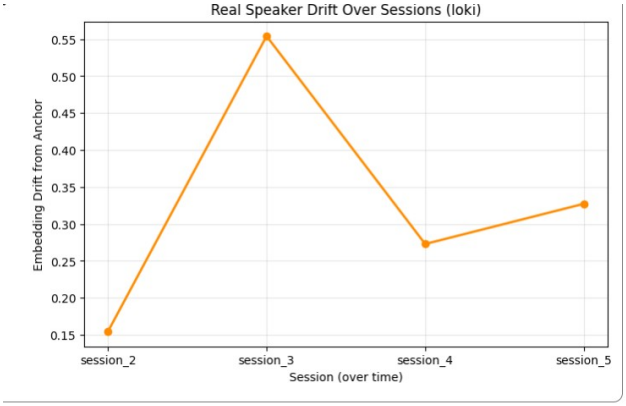



**Figure 5.3 – Embedding drift from a fixed anchor across real recordings**

These results show that the drift signal responds in the expected direction to controlled degradation and reveal a real complication, session-level variability, which the full evaluation must take into account. They do not yet constitute the completed experimental evaluation: the full word-error-rate and mean-opinion-score experiments on TORGO are in progress.

# CHAPTER 6
# CONCLUSION AND FUTURE WORK

EchoGenesis presents a novel idea for preserving one's identity during the progressive loss of the ability to speak. The framework uses an earlier, healthy recording to capture a person's speaker characteristics with a degradation-robust speaker encoder. Later, degraded samples are compared to this reference, and not the other way round, which allows the identity reference to be maintained while the voice changes. The few-shot TTS stage is applied to the resulting representation to synthesize speech from text, with prosody-aware chunked synthesis used for longer utterances.

A working prototype was implemented with a web interface and a FastAPI backend. The backend provides a longitudinal voice bank with similarity and drift tracking, a phrase bank of real recordings for emergency and common phrases, and cached speech synthesis in a fine-tuned XTTS-v2 voice. The proof-of-concept experiments showed that the embedding drift signal responds in the expected direction to controlled degradation (increasing from 0 to roughly 0.22 as speech was slowed by 50%) and that different degradation types affect the embedding space unevenly (slowed and slurred speech about 0.31 and 0.30, noisy speech about 0.10). A study across real recording sessions also showed that session-to-session variability can be larger than the gradual voice change being tracked, which the final evaluation must account for.

The approach is intended for patients with ALS, Parkinson's disease, stroke, dysarthria or other conditions that degrade speech or leave them mute after surgery. The full evaluation of speaker similarity, word error rate and degradation robustness on the TORGO dataset is in progress and will complete the experimental validation of the framework.

**Key Achievements**

- A longitudinal voice-preservation workflow that keeps the healthy voice as the identity reference.
- An embedding-drift measure that responds to controlled degradation.
- A usable, accessible web interface backed by a FastAPI inference server.
- Prosody-aware chunked synthesis for long sentences.

The future enhancement of this project "EchoGenesis" can be improved by evaluating voice quality using the TORGO dataset, handling session variations, and fine-tuning XTTS-v2 for female voices. Future development includes Tamil, Hindi, and Malayalam support, a mobile application, scalable storage, speech enhancement, and clinical validation with patients and caregivers.

## REFERENCES:


[1] R. Vinotha, D. Hepsiba, L. D. Vijay Anand and D. J. Reji, “Empowering communication: Speech technology for Indian and western accents through AI-powered speech synthesis,” arXiv:2401.11771v2, 2024.

[2] M. Anuprabha, K. Gurugubelli and A. K. Vuppala, “Fairness in dysarthric speech synthesis: Understanding intrinsic bias in dysarthric speech cloning using F5-TTS,” arXiv:2508.05102v3, 15 Aug. 2025.

[3] B. Moell and F. S. Aronsson, “Voice cloning for dysarthric speech synthesis: Addressing data scarcity in speech-language pathology,” arXiv:2503.01266v1, 3 Mar. 2025.

[4] Y. Jia, Y. Zhang, R. J. Weiss, Q. Wang, J. Shen, F. Ren, Z. Chen, P. Nguyen, R. Pang, I. L. Moreno and Y. Wu, “Transfer learning from speaker verification to multispeaker text-to-speech synthesis,” arXiv:1806.04558v4 [cs.CL], 2 Jan. 2019.

[5] G. Ruggiero, E. Zovato, L. Di Caro and V. Pollet, “Voice cloning: A multi-speaker text-to-speech synthesis approach based on transfer learning,” arXiv:2102.05630v1 [cs.SD], 10 Feb. 2021.

[6] S. Singh, Q. Wang, Z. Zhong, C. Mendes, M. Hasegawa-Johnson, W. Abdulla and S. R. Shahamiri, “Low-burden data augmentation for dysarthric ASR via zero-shot voice cloning,” arXiv:2606.19823v1, 18 Jun. 2026.

[7] S. Lee, S. Han, S. Cheong and J. W. Shin, “FlowSE: Flow matching-based speech enhancement,” arXiv:2508.06840v1 [eess.AS], 9 Aug. 2025.

[8] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” arXiv:2212.04356v1 [eess.AS], 6 Dec. 2022.

[9] L. Lu, L. Kong, C. Dyer and N. A. Smith, “Multitask learning with CTC and segmental CRF for speech recognition,” arXiv:1702.06378v4 [cs.CL], 5 Jun. 2017.

[10] L. Wan, Q. Wang, A. Papir and I. L. Moreno, “Generalized end-to-end loss for speaker verification,” arXiv:1710.10467, 2018.

[11] J. Shen, R. Pang, R. J. Weiss, M. Schuster, N. Jaitly, Z. Yang, Z. Chen, Y. Zhang, Y. Wang, R. Skerry-Ryan, R. A. Saurous, Y. Agiomyrgiannakis and Y. Wu, “Natural TTS synthesis by conditioning WaveNet on mel spectrogram predictions,” arXiv:1712.05884, 2018.

[12] S. Ö. Arik, J. Chen, K. Peng, W. Ping and Y. Zhou, “Neural voice cloning with a few samples,” arXiv:1802.06006, 2018.

[13] C. Veaux, J. Yamagishi and S. King, “Towards personal synthesized voices for individuals with vocal disabilities: Voice banking and reconstruction,” in Proc. 4th Workshop on Speech and Language Processing for Assistive Technologies (SLPAT), Grenoble, France, 2013, pp. 107–111.

[14] S. Regondi, G. Donvito, E. Frontoni, M. Kostovic, F. Minazzi, S. Bratieres, M. Filosto and R. Pugliese, “Artificial intelligence empowered voice generation for amyotrophic lateral sclerosis patients,” Scientific Reports, vol. 15, no. 1, 2025.

[15] F. Rudzicz, A. K. Namasivayam and T. Wolff, “The TORGO database of acoustic and articulatory speech from speakers with dysarthria,” Language Resources and Evaluation, vol. 46, no. 4, pp. 523–541, 2012.

SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

# CERTIFICATE OF PARTICIPATION

This certificate is presented to

S. UPENDIRAN

Vels University

for actively participating in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

Voice Beyond Silence : Echogenesis · A longitudinal degradation-Robust framework for voice preservation in Progressive Speech loss

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
**Dr. B. Ramesh**

SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

# CERTIFICATE OF PARTICIPATION

This certificate is presented to

SAIYUJ · S

Vels University

for actively participating in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

Voice Beyond Silence : Echogenesis · A longitudinal degradation-Robust framework for voice preservation in Progressive Speech loss.

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
**Dr. B. Ramesh**

SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

# CERTIFICATE OF PARTICIPATION

This certificate is presented to

B. Lokesh

Vels University

for actively participating in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

Voice Beyond Silence : Echogenesis. A longitudinal degradation-Robust framework for voice preservation in Progressive Speech Loss.

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
**Dr. B. Ramesh**

SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

# CERTIFICATE OF PARTICIPATION

This certificate is presented to

Dr. G. Revathy

Vels University

for actively participating in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

Voice Beyond Silence : Echogenesis. A longitudinal degradation-Robust framework for voice preservation in Progressive Speech Loss.

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
**Dr. B. Ramesh**